

AI.pdf - ECE 422C Assignment 2

Name: Mayank Konduri

Assignment: Mastermind - Phase I + Wordle - Phase II

1. AI Tools Used

I used **ChatGPT** as I was working through this assignment.

2. AI Interaction 1 - “Second Set of Eyes”

I state this interaction as the first on this list as the majority of my AI usage was pasting my code in the free ChatGPT browser version in order to make sure that my syntax was correct. When coding fast, I usually misspell variables, inaccurately define certain access specifiers, and mess up on closing brackets. To optimize my debugging efforts on test-cases, I used ChatGPT in order to go through my final code and make sure there were no major syntax errors, and I fixed my code given the feedback, and NEVER pasted code directly into my codebase. In this process, I was also able to rename certain variable names with the convention that ChatGPT suggested.

Continuing on from how I used AI in Phase I, I designed the structure of classes myself, including how I wanted to deduce the work into abstract classes + child classes, and then after I finished coding, I gave AI my code, and initially asked for any syntax or risky compilation errors that could be fixed with edge-case checking. Next, I gave AI my intended plan for the code, and how I envisioned to introduce abstract classes and child classes, to allow for a more adaptable game. AI then gave me suggestions on where I succeeded, and very nicely, gave me suggestions on how I could take it one step further and ‘give more work’ to the parent class, which was very interesting. That’s how both the feedback classes for both the games ended up being 17 lines and 26 lines (even with a lot of comments).

3. AI Interaction 2 - “Testing Companion”

The second way I used ChatGPT as my AI tool was after I finished coding the entire project, and went through a few test runs by myself. After I decided that the code was working as per the instruction sheet, pasted the instructions to ChatGPT, and **without** pasting my code (and just letting the AI know how input is expected), I asked it to provide several test cases. I was in the ‘test-mode,’ so the AI knew the secret code, and therefore was able to give me test cases like duplicates, input error, and win logic. This is how I gained confidence that my solution was error-proof to the complex inputs that the Gradescope autograder might use.

The same thing also applied to using AI to help generate test cases. I think after a while (and a lot of coding), it gets really tedious to go through all test cases, and you have to actually guess the words, and try all inputs, and this is where AI is really helpful. I gave it the instructions Dr. Thomaz provided for the Wordle game, and gave it how the game was supposed to start “java assignment2.Driver wordle test,” and then asked AI to give me like 30 different things to test out, and what inputs to put and what outputs to expect. That was super helpful, as within the span of 15 minutes, I made like 3 key changes to my code which definitely would have taken me another hour to debug myself (if I was ever able to figure out that there was an error in the first place).

4. AI Interaction 3 - “Terminal Commands”

I basically resorted to ChatGPT's help for most of the commands to create a .zip file without the .class files and only the .java files. ChatGPT told me what directory to go to, and what files to make sure were in them. Once confirmed, it was able to give me a command to generate a .zip file, and then I was able to access the .zip file and submit it to Gradescope for initial testing of my solution. Therefore, I used ChatGPT for most of the terminal commands.

I believe AI was even more crucial in the development process of Phase II because of the usage of an external data source. Because of this, I had a Dictionary.txt, and I had then needed to have a DictionaryHelper.java which was responsible for converting the raw text into a data structure. Because of having to work with file paths, and processing a .txt file, I used AI a lot to learn about 'FileNotFoundException', and how to use the Scanner class to go through a text file. Without AI, I would have done the research online, but since I was able to use AI, I asked it to quickly teach me how file I/O works, and I coded all the .txt parsing by myself. I did not use AI to code the parsing algorithm, but I definitely used AI to learn how file parsing works, and how to include the try-catch statements to avoid any compile-time or run-time errors.

5. AI-Generated Suggestion I Changed

One last way that I used AI was in the very beginning when I pasted the instructions to the ChatGPT web version, and then walked through what I understood of the assignment. After a few back and forth (not in the code phase, and just ideation phase), I sent a really big paragraph of what classes I plan to have, and how I plan to structure my code. The part I actually changed in my initial implementation came when ChatGPT told me that instead of having a Mastermind.java class with all the functions initially defined, it would be smart to have a GameTemplate.java which acts like an abstract class and has all the method to be implemented declared. So that whenever we have the main interface, we can have an object of the game type, but declare it as GameTemplate so that all the subsequent code can be based off the polymorphism concept, and the online line to be changed for each game is the reference part of the object instantiation. I thought that structural suggestion was very useful, and I changed my plans according to that.

As I was making my classes more optimized so that there was one abstract class, and 2 child classes, I asked AI if there were any other shared classes that could be optimized in such a way. AI suggested that I have a 'ConfigurationTemplate' and then have 'MastermindConfiguration' and 'WordleConfiguration' as the child classes that overridden the methods from the parent classes. I tried writing out the classes, and noticed that there was a lot of data and methods specific to the game's configuration, and the parent class only ended up being a couple of lines, so I decided that this symmetry was not worth a parent class that was super small, and that each new game should have its own configuration.